

GESTOCK

Diagramas de Arquitectura e Integración: CLIENTE -> MID -> API



18 CONTROLADORES REALES (un diagrama por pagina):
auth usuario rol permiso empresa
categoria bodega producto movimiento recepcion
auditoria incidencia mantenimiento sesion notificacion
configuracion dashboard reporte

Cada pagina muestra: flujo CLIENTE->MID->API, tabla de rutas con permisos y middleware, capa de datos (router->controller->service/repo->PostgreSQL) y codigos de error posibles.

paginas transversales: arquitectura, MID, autenticacion, RBAC, CRUD, errores, BD, matriz, ciclo ida/vuelta, retorno sin MID, publicos sin MID, pipeline app, rutas frontend, mapa paginas, errores cli

Estilo Paint: fondo blanco, trazo negro, figuras 2D (sin sombras ni 3D).
Información basada exclusivamente en el código fuente real del proyecto (rutas, permisos, middleware, tablas y errores verificados en backend/src y frontend_gestock/src).

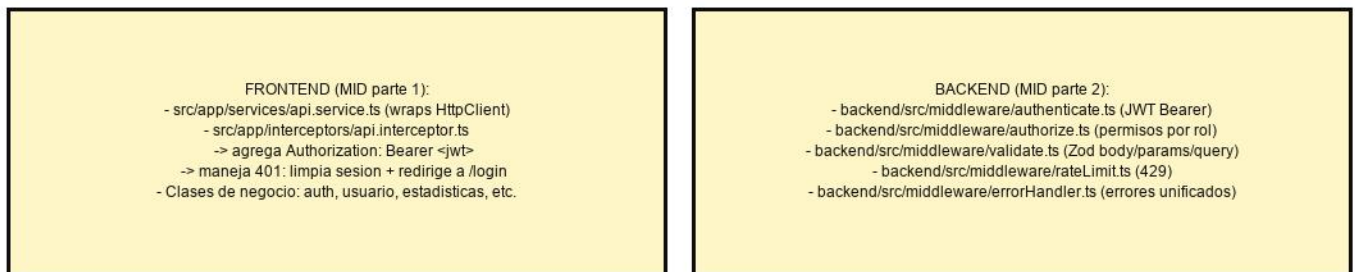
ARQUITECTURA GENERAL GESTOCK

CLIENTE (Angular) -> MID (front+back) -> API (Express) -> PostgreSQL

VISION DE ALTO NIVEL: CLIENTE -> MID -> API



DETALLE DEL MID (no ficticio)



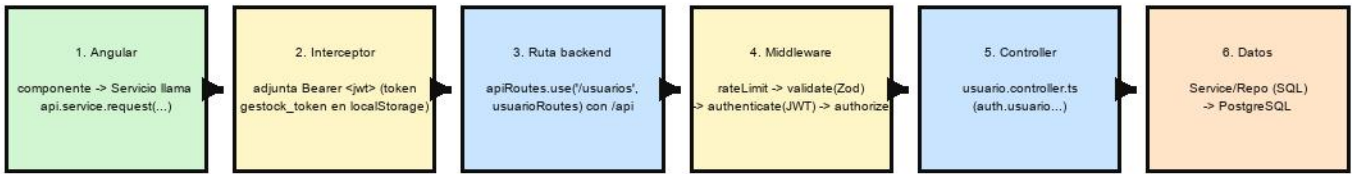
18 CONTROLADORES REALES (routes/ de backend/src)

auth usuario rol permiso empresa categoria bodega producto movimiento recepcion
auditoria incidencia mantenimiento sesion notificacion configuracion dashboard reporte

LA CAPA MID EN GESTOCK (DETALLE)

Frontera de seguridad entre el navegador y los controladores

FLUJO DE UNA PETICION AUTENTICADA



ARCHIVOS REALES DE LA CAPA MID

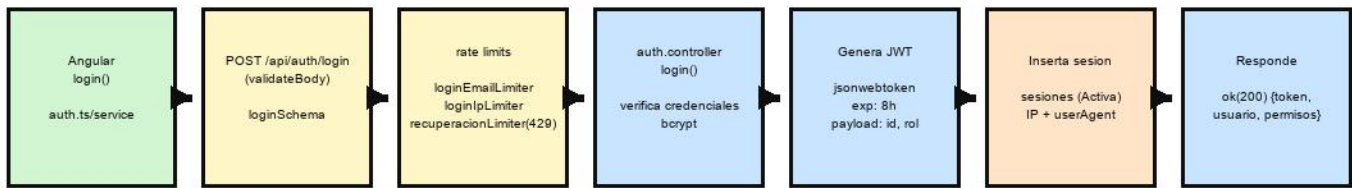
FRONTEND (Interceptores y Guards):
src/app/interceptors/api.interceptor.ts
src/app/guards/*.guard.ts (canActivate)
src/app/services/api.service.ts
src/app/services/*.service.ts
src/app/routes.ts (rutas protegidas)

BACKEND (Middleware):
backend/src/middleware/authenticate.ts
backend/src/middleware/authorize.ts
backend/src/middleware/validate.ts
backend/src/middleware/rateLimit.ts
backend/src/middleware/errorHandler.ts

AUTENTICACION Y SESION

Login con rateLimit -> JWT 8h -> sesiones activas -> logout

POST /auth/login (auth.routes.ts)



RUTAS PROTEGIDAS CON MIDDLEWARE

GET /auth/perfil | POST /auth/logout | PUT /auth/contrasena -> authenticateToken (valida JWT + sesion Activa)
POST /auth/recuperar -> solo rateLimit + validateBody (envio de correo con Nodemailer)
POST /auth/recuperar/validar -> valida el codigo generado
POST /auth/recuperar/restablecer -> hashsea nueva contrasena

FUNCIONAMIENTO EN FRONTEND (MID)

Tras login: localStorage guarda 'gestock_token' y 'gestock_usuario_sesion'.
api.interceptor.ts agrega Authorization: Bearer <token> a todas las peticiones.
EN 401 (token invalido o sesion cerrada): limpia la sesion local,
muestra toast 'Tu sesion ha expirado' y redirige a /auth/login.
Guards (permission-guard / role-guard / home-guard) protegen cada ruta.

CONTROL DE ACCESO RBAC (Roles + Permisos)

roles -< rol_permiso >- permisos - usuarios.n Rol

MODELO DE DATOS RBAC

usuarios

rol_id INT
-> roles(id)
estado
empresa_id

roles

nombre UNIQUE
estado
descripcion

rol_permiso

rol_id
permiso_id
PK (rol_id,permiso_id)

permisos

codigo UNIQUE
modulo
nombre

FLUJO DE AUTORIZACION EN EL MID (authorize.ts)

Request

con Bearer + usuario

authenticate

verifica JWT y
usuario Activo
+ sesion Activa

authorizePermission

busca permiso del rol
ej. 'usuarios.view'

SIN PERMISO

403 FORBIDDEN

CON PERMISO

continua al controller
-> service/repo

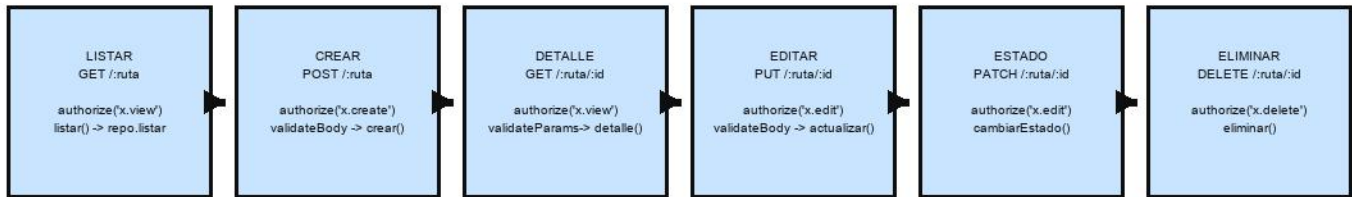
EJEMPLO DE CODIGO (repollo fisico en rutas)

```
usuarioRoutes.get('/', authorizePermission('usuarios.view'), usuarioController.listar);
usuarioRoutes.patch('/:id/estado', authorizePermission('usuarios.desactivar'), ...);
rolRoutes.put('/:id/permisos', authorizePermission('roles.asignar_permisos'), validateBody(asignarPermisosSchema), ...);
recepcionRoutes.get('/', authorizeAnyPermission('recepcion.view', 'historial.view'), ...);
```

PATRON CRUD TIPO EN GESTOCK

Un controlador = un modulo · Create/Read/Update/Delete con auditoria

CICLO DE VIDA DE UN REGISTRO (ej. Producto)



CODIGO TIPO EN UN CONTROLLER (usuario.controller.ts)

```
async listar(req, res, next) { try { const lista = await usuarioService.listar(req.usuarioId); ok(res, lista); }  
  catch (e) { next(e); } }  
async crear(req, res, next) { try { const nuevo = await usuarioService.crear(req.body); created(res, nuevo); }  
  catch (e) { next(e); } }
```

MANEJO DE ERRORES UNIFICADO

ApiError.ts + errorHandler.ts (middleware) -> respuesta JSON {code, message}

CODIGOS DE ESTADO ECUPCIOLADOS POR ApiError

Codigo	Code (ApiError/DB)	Significado
400	BAD_REQUEST · NOT_NULL_VIOLATION(23502) INVALID_PARAMETER(22P02) · BAD_JSON	DTO invalido, campos requeridos, JSON roto
401	UNAUTHORIZED	token inexistente/expirado, sesion cerrada
403	FORBIDDEN	sin permiso para la accion
404	NOT_FOUND	recurso no existe
409	CONFLICT · DUPLICATE_KEY(23505) FOREIGN_KEY_VIOLATION(23503)	registro duplicado / FK violada
429	RATE_LIMITED	demasiadas solicitudes
500	INTERNAL_ERROR	error no controlado

FLUJO: el error viaja al errorHandler (app.ts)

controller -> next(err) -> errorHandler.ts
se normaliza el error de BD (constraints de postgres) en codigo interno
respuesta: HTTP <status> { code: '...', message: '...', details? }
el frontend api.interceptor.ts muestra el toast y en 401 redirige a /login

BASE DE DATOS GESTOCK (PostgreSQL)

18 tablas + relaciones (multitenant: empresa_id en tablas de negocio)

TABLAS REALES DE GESTOCK

NUCLEO / IDENTIDAD

empresas · roles · permisos · rol_permiso · usuarios · sesiones

OPERACIONES

incidencias · mantenimientos

CATALOGOS

categorias · bodegas · productos

SEGURIDAD

auditorias · notificaciones · restablecimientos_contrasena

INVENTARIO

movimientos_inventario · recepciones · recepcion_detalle

CONFIG

configuracion_sistema

RELACIONES PRINCIPALES

usuarios.rol_id -> roles.id | usuarios.empresa_id -> empresas.id
rol_permiso(rol_id, permiso_id) -> roles / permisos
productos.categoria_id -> categorias.id | productos.bodega_id -> bodegas.id
movimientos_inventario.producto_id -> productos.id
recepciones.usuario_id -> usuarios.id | recepcion_detalle.recepcion_id -> recepciones.id

MATRIZ DE TRAZABILIDAD: CONTROLADOR -> MODULO -> PERMISOS

Resumen de los 18 controladores reales

Controlador	Modulo	Rutas (metodo)	Tablas BD
auth	Auth	POST /auth/login, GET /auth...	usuarios · sesiones · restablecim...
usuario	Usuarios	GET /usuarios, GET /usuario...	usuarios · roles · empresas
rol	Roles	GET /roles/basicos, GET /ro...	roles · rol_permiso · permisos · ...
permiso	Permisos	GET /permisos, GET /permiso...	permisos
empresa	Empresas	POST /empresas/seleccionar,...	empresas
categoria	Categorias	GET /categorias, GET /categ...	categorias
bodega	Bodegas	GET /bodegas, GET /bodegas/...	bodegas
producto	Productos	GET /productos, GET /produc...	productos · categorias · bodegas
movimiento	Movimientos de Inventario	GET /movimientos, GET /movi...	movimientos_inventario · producto...
recepcion	Recepcion de Mercancia	GET /recepciones, GET /rece...	recepciones · recepcion_detalle · ...
auditoria	Auditoria	GET /auditorias, GET /audit...	auditorias · usuarios
incidencia	Incidencias	GET /incidencias, GET /inci...	incidencias · usuarios
mantenimiento	Mantenimiento	GET /mantenimientos, GET /m...	mantenimientos · usuarios
sesion	Sesiones	GET /sesiones, POST /sesion...	sesiones · usuarios
notificacion	Notificaciones	GET /notificaciones, PATCH ...	notificaciones
configuracion	Configuracion del Sistema	GET /configuracion, PUT /co...	configuracion_sistema
dashboard	Panel Principal	GET /dashboard/resumen	productos · movimientos_inventari...
reporte	Reportes	GET /reportes/inventario, G...	productos · movimientos_inventari...

CONTROLADOR: AUTH

CLIENTE -> MID -> API · Autenticación / Auth



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
POST	/auth/login	validateBody(loginSchema) + loginEmailLimiter + loginIp...
GET	/auth/perfil	authenticateToken
POST	/auth/logout	authenticateToken
PUT	/auth/contrasena	authenticateToken + validateBody(cambiarContrasenaSchema)
POST	/auth/recuperar	recuperacionLimiter + validateBody(solicitarRecuperacio...
POST	/auth/recuperar/validar	validateBody(validarCodigoRecuperacionSchema)
POST	/auth/recuperar/restablecer	validateBody(restablecerPasswordSchema)

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 credenciales/token invalidos	403 usuario inactivo
404 usuario no existe	409 contraseña actual incorrecta	429 rate limit
500 error interno		

CONTROLADOR: USUARIO

CLIENTE -> MID -> API · Usuarios / Usuarios



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/usuarios	authorizePermission('usuarios.view')
GET	/usuarios/:id	authorizePermission('usuarios.view') + validateParams
POST	/usuarios	authorizePermission('usuarios.create') + validateBody
PUT	/usuarios/:id	authorizePermission('usuarios.edit') + validateBody + v...
PATCH	/usuarios/:id/estado	authorizePermission('usuarios.desactivar')
PATCH	/usuarios/:id/rol	authorizePermission('usuarios.edit') + validateBody
PUT	/usuarios/:id/password	authorizePermission('usuarios.password') + validateBody
DELETE	/usuarios/:id	authorizePermission('usuarios.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 usuario no existe	409 email duplicado / FK	500 error interno

CONTROLADOR: ROL

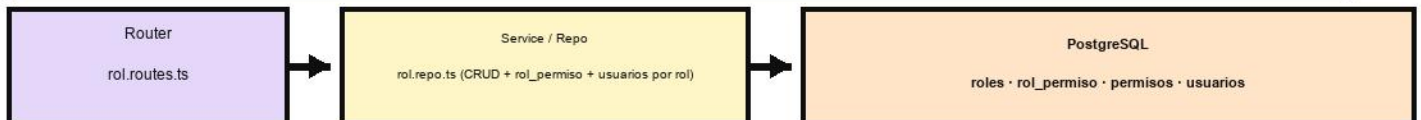
CLIENTE -> MID -> API · Roles / Roles



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/roles/basicos	authorizePermission('roles.view')
GET	/roles	authorizePermission('roles.view')
GET	/roles/:id	authorizePermission('roles.view') + validateParams
POST	/roles	authorizePermission('roles.create') + validateBody
PUT	/roles/:id	authorizePermission('roles.edit') + validateBody
PATCH	/roles/:id/estado	authorizePermission('roles.edit') + validateBody
PUT	/roles/:id/permisos	authorizePermission('roles.asignar_permisos') + validat...
DELETE	/roles/:id	authorizePermission('roles.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 rol no existe	409 nombre duplicado / FK usuarios	500 error interno

CONTROLADOR: PERMISO

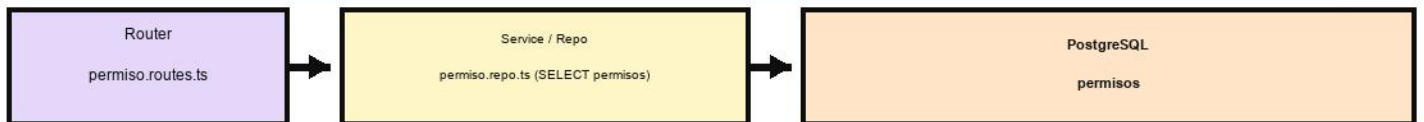
CLIENTE -> MID -> API · Permisos / Permisos



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/permisos	authorizePermission('permisos.view')
GET	/permisos/modulo/:modulo	authorizePermission('permisos.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)



CONTROLADOR: EMPRESA

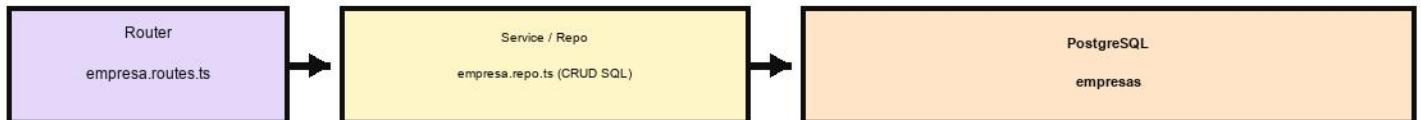
CLIENTE -> MID -> API · Empresas / Empresas



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
POST	/empresas/seleccionar	authorizePermission('empresas.view') + validateBody
GET	/empresas	authorizePermission('empresas.view')
GET	/empresas/:id	authorizePermission('empresas.view') + validateParams
POST	/empresas	authorizePermission('empresas.create') + validateBody
PUT	/empresas/:id	authorizePermission('empresas.edit') + validateBody
DELETE	/empresas/:id	authorizePermission('empresas.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 empresa no existe	409 NIT duplicado / FK	500 error interno

CONTROLADOR: CATEGORIA

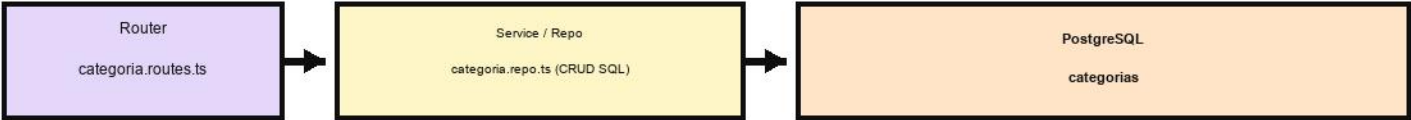
CLIENTE -> MID -> API · Categorías / Categorías



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/categorias	authorizePermission('categorias.view')
GET	/categorias/:id	authorizePermission('categorias.view') + validateParams
POST	/categorias	authorizePermission('categorias.create') + validateBody
PUT	/categorias/:id	authorizePermission('categorias.edit') + validateBody
DELETE	/categorias/:id	authorizePermission('categorias.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

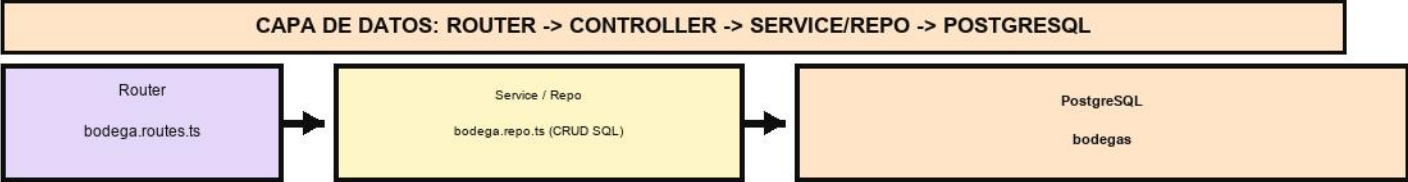
400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 categoria no existe	409 nombre duplicado / FK	500 error interno

CONTROLADOR: BODEGA

CLIENTE -> MID -> API · Bodegas / Bodegas



RUTAS, PERMISOS Y MIDDLEWARE (reales)		
Metodo	Ruta	Permiso / Middleware
GET	/bodegas	authorizePermission('bodegas.view')
GET	/bodegas/:id	authorizePermission('bodegas.view') + validateParams
POST	/bodegas	authorizePermission('bodegas.create') + validateBody
PUT	/bodegas/:id	authorizePermission('bodegas.edit') + validateBody
DELETE	/bodegas/:id	authorizePermission('bodegas.delete') + validateParams



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)		
400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 bodega no existe	409 nombre/codigo duplicado / FK	500 error interno

CONTROLADOR: PRODUCTO

CLIENTE -> MID -> API · Productos / Productos



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/productos	authorizePermission('productos.view')
GET	/productos/stock-bajo	authorizePermission('inventario.view')
GET	/productos/:id	authorizePermission('productos.view') + validateParams
POST	/productos	authorizePermission('productos.create') + validateBody
PUT	/productos/:id	authorizePermission('productos.edit') + validateBody
PATCH	/productos/:id/estado	authorizePermission('productos.edit') + validateBody
DELETE	/productos/:id	authorizePermission('productos.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 producto no existe	409 codigo duplicado / FK	500 error interno

CONTROLADOR: MOVIMIENTO

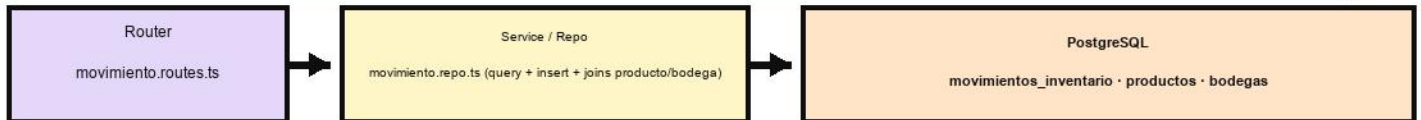
CLIENTE -> MID -> API · Movimientos / Movimientos de Inventario



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/movimientos	authorizePermission('movimientos.view') + validateQuery
GET	/movimientos/:id	authorizePermission('movimientos.view') + validateParams
POST	/movimientos	authorizePermission('recepcion.create') + validateBody
DELETE	/movimientos/:id	authorizePermission('recepcion.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 movimiento no existe	409 FK producto/bodega	500 error interno

CONTROLADOR: RECEPCION

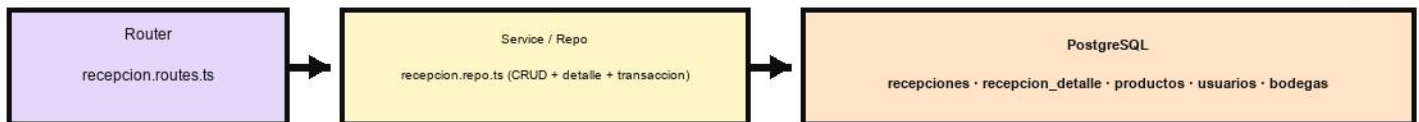
CLIENTE -> MID -> API · Recepciones / Recepcion de Mercancia



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/recepciones	authorizeAnyPermission('recepcion.view','historial.view')
GET	/recepciones/:id	authorizeAnyPermission('recepcion.view','historial.view...')
POST	/recepciones	authorizePermission('recepcion.create') + validateBody
PUT	/recepciones/:id	authorizePermission('recepcion.edit') + validateBody
DELETE	/recepciones/:id	authorizePermission('recepcion.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 recepcion no existe	409 numero_documento duplicado / FK	500 error interno

CONTROLADOR: AUDITORIA

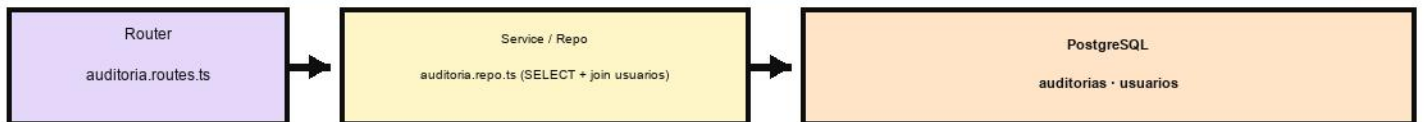
CLIENTE -> MID -> API · Auditorias / Auditoria



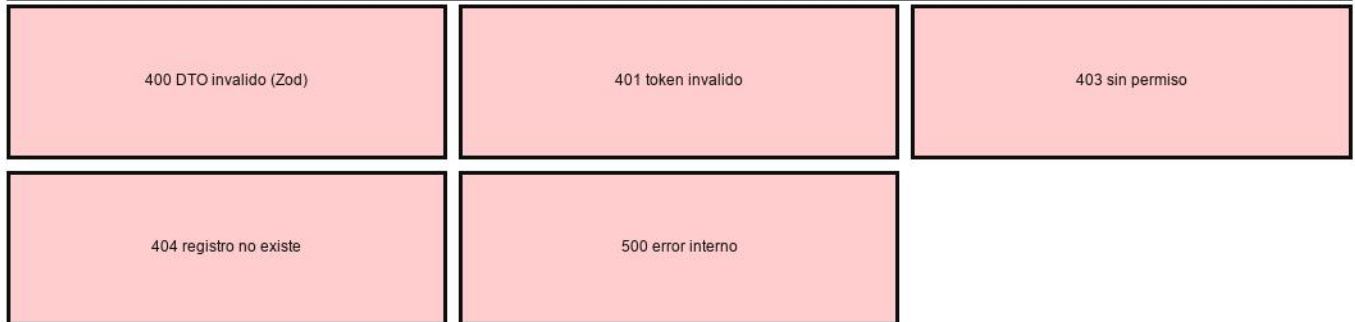
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/auditorias	authorizePermission('auditoria.view')
GET	/auditorias/:id	authorizePermission('auditoria.view') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)



CONTROLADOR: INCIDENCIA

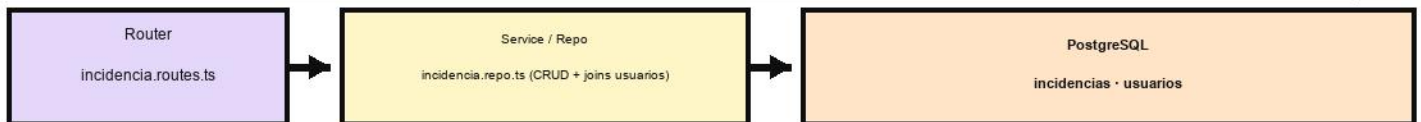
CLIENTE -> MID -> API · Incidencias / Incidencias



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/incidencias	authorizePermission('incidencias.view')
GET	/incidencias/:id	authorizePermission('incidencias.view') + validateParams
POST	/incidencias	authorizePermission('incidencias.create') + validateBody
PUT	/incidencias/:id	authorizePermission('incidencias.edit') + validateBody
DELETE	/incidencias/:id	authorizePermission('incidencias.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

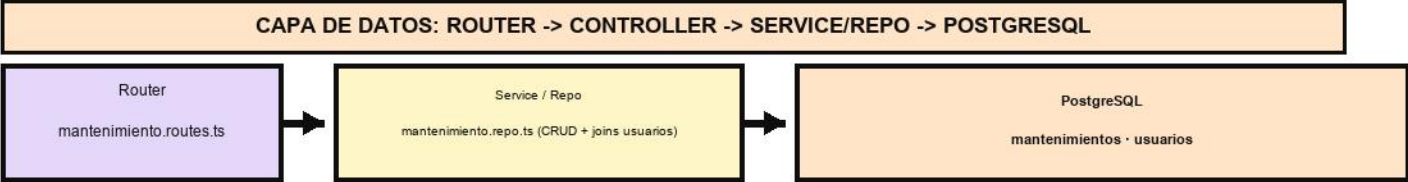
400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 incidencia no existe	409 FK usuarios	500 error interno

CONTROLADOR: MANTENIMIENTO

CLIENTE -> MID -> API · Mantenimientos / Mantenimiento



RUTAS, PERMISOS Y MIDDLEWARE (reales)		
Metodo	Ruta	Permiso / Middleware
GET	/mantenimientos	authorizePermission('mantenimiento.view')
GET	/mantenimientos/:id	authorizePermission('mantenimiento.view') + validateParams
POST	/mantenimientos	authorizePermission('mantenimiento.create') + validateBody
PUT	/mantenimientos/:id	authorizePermission('mantenimiento.edit') + validateBody
DELETE	/mantenimientos/:id	authorizePermission('mantenimiento.delete') + validateP...



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)		
400 DTO invalido (Zod)	401 token invalido	403 sin permiso
404 mantenimiento no existe	500 error interno	

CONTROLADOR: SESION

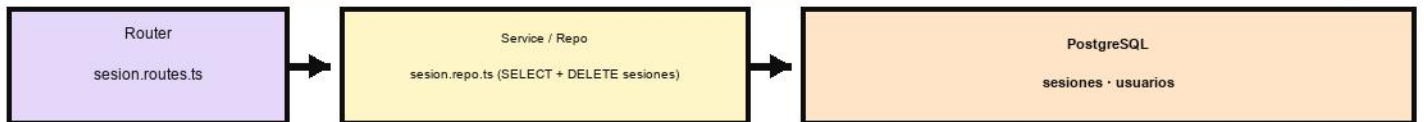
CLIENTE -> MID -> API · Sesiones / Sesiones



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/sesiones	authorizePermission('sesiones.view')
POST	/sesiones/cerrar-otras	authorizePermission('sesiones.delete')
DELETE	/sesiones/:id	authorizePermission('sesiones.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)



CONTROLADOR: NOTIFICACION

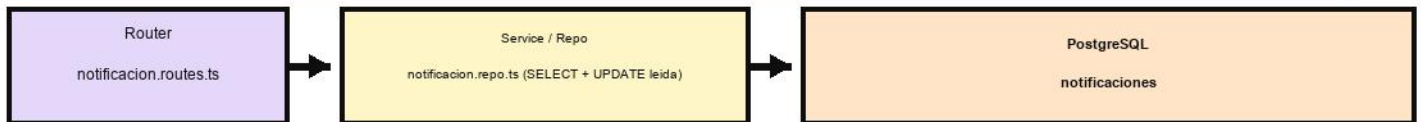
CLIENTE -> MID -> API · Notificaciones / Notificaciones



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/notificaciones	authorizePermission('notificaciones.view')
PATCH	/notificaciones/:id/leida	authorizePermission('notificaciones.view') + validateBody
POST	/notificaciones/leidas	authorizePermission('notificaciones.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)



CONTROLADOR: CONFIGURACION

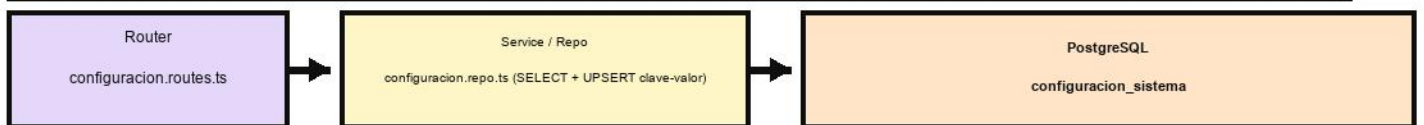
CLIENTE -> MID -> API · Configuración / Configuración del Sistema



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/configuracion	authorizePermission('configuracion.view')
PUT	/configuracion	authorizePermission('configuracion.edit') + validateBody

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)

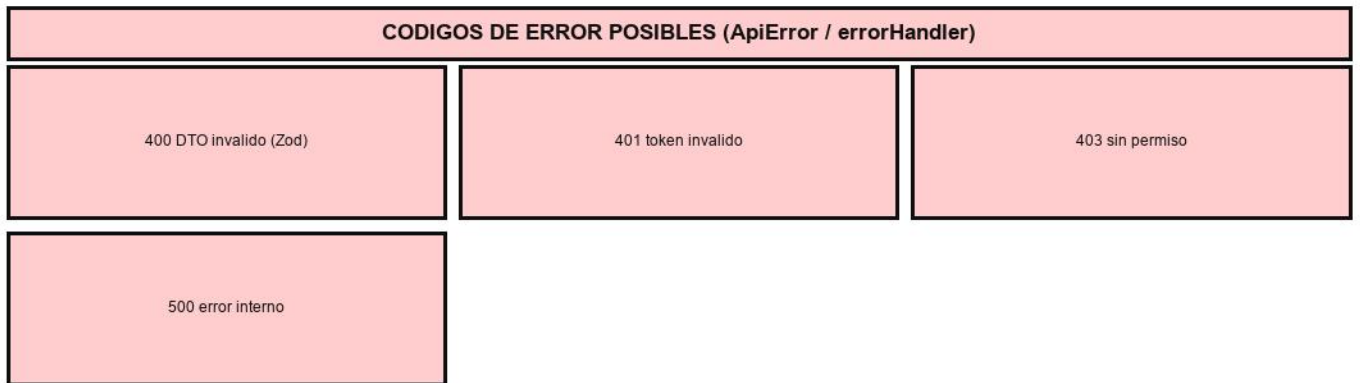
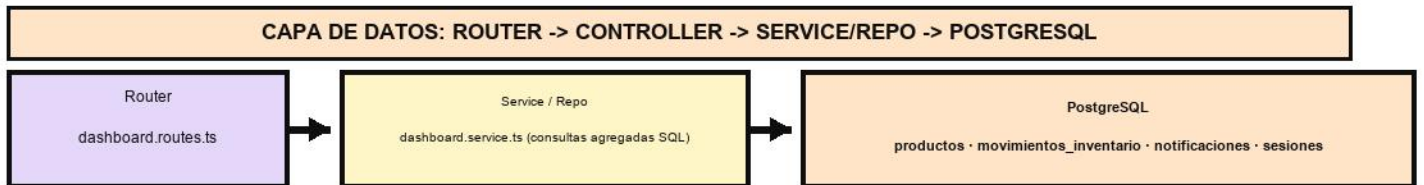


CONTROLADOR: DASHBOARD

CLIENTE -> MID -> API - Dashboard / Panel Principal



RUTAS, PERMISOS Y MIDDLEWARE (reales)		
Metodo	Ruta	Permiso / Middleware
GET	/dashboard/resumen	authorizePermission('dashboard.view')



CONTROLADOR: REPORTE

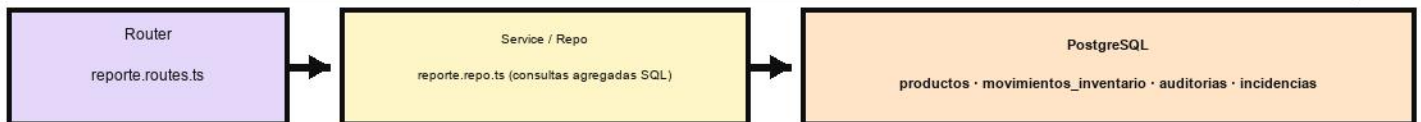
CLIENTE -> MID -> API · Reportes / Reportes



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/reportes/inventario	authorizePermission('reportes.view')
GET	/reportes/movimientos	authorizePermission('reportes.view')
GET	/reportes/stock	authorizePermission('reportes.view')
GET	/reportes/auditorias	authorizePermission('reportes.view')
GET	/reportes/incidencias	authorizePermission('reportes.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



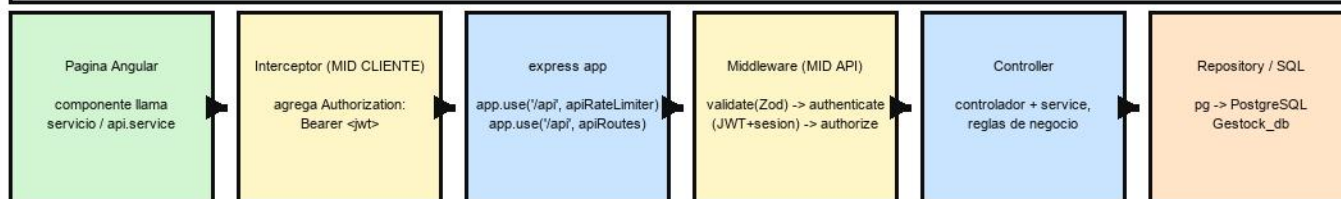
CODIGOS DE ERROR POSIBLES (ApiError / errorHandler)



CICLO COMPLETO DE UNA PETICION: IDA CON MID, VUELTA SIN MID

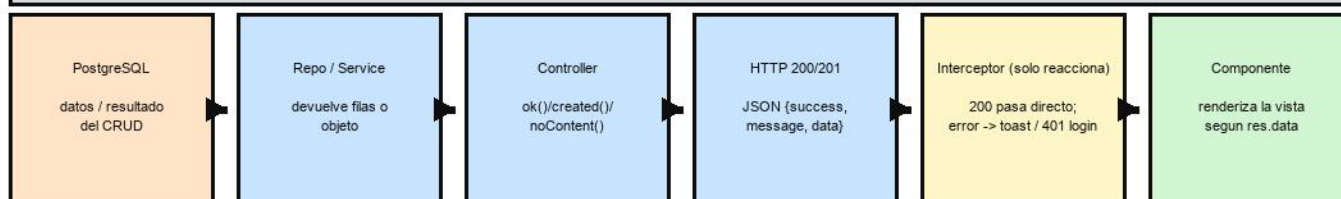
El MID valida la petición que ENTRA; la respuesta SALE directo del API al cliente

1) IDA (SOLICITUD) - EL MID INTEGRRA AMBOS LADOS



El MID NO revalida nada en la vuelta: la respuesta no vuelve a pasar por authenticate/authorize/validate.

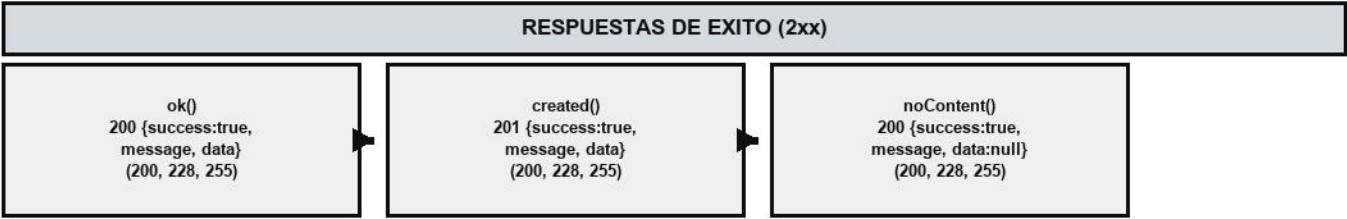
2) VUELTA (RESPUESTA) - CRUD -> CLIENTE SIN VOLVER POR EL MID



REGLA REAL DE LA ARQUITECTURA:
IDA -> el interceptor (frontend) + validate/authenticate/authorize (backend) forman el MID.
VUELTA -> la respuesta viaja directa del controller al cliente. El interceptor solo la observa: si es error (catchError), muestra toast y en 401 cierra la sesion. El backend no vuelve a validar los datos que ya genero.

RETORNO API -> CLIENTE SIN MID (CRUD DIRECTO)

Solo los errores y el 401 reaccionan en el interceptor



Estas respuestas NO pasan por ninguna validacion del MID: llegan tal cual al componente.

RESPUESTAS DE ERROR (4xx/5xx) - MANEJADAS POR EL INTERCEPTOR (MID CLIENTE)	
DONDE	QUE PASA EN EL CLIENTE
Interceptor	401 != login -> limpia sesion + toast + /auth/login
Interceptor	otro status != 401 -> toast(message, 'error')
Interceptor	login (401/400) -> NO redirige; login.ts muestra 'Credenciales incorrectas'
Guard	no autenticado -> navega /auth/login; sin permiso -> ruta inicial por rol
Pagina	catch local -> toast error personalizado

CONCLUSION: el MID (defensa) se aplica SIEMPRE en la ida. La vuelta es directa entre el API y el cliente; el interceptor solo espia la respuesta para UX (toasts) o recuperacion de sesion (401). Ningun dato de vuelta es re-validado por el backend.

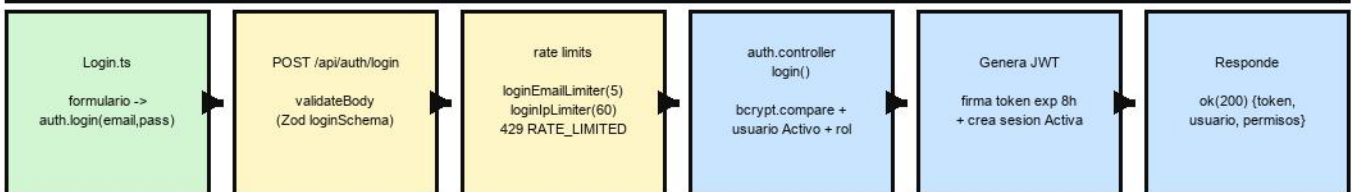
ENDPOINTS PUBLICOS: FLUJOS QUE NO REQUIEREN MID

Login, recuperacion y documentacion acceden sin token (sin authenticate/authorize)

RUTAS QUE RESPONDEN AL CLIENTE SIN PASAR POR AUTHENTICATE/AUTHORIZE

Metodo	Ruta	Middleware que usa (sin authenticate)
GET	/	info de la API (version, docs, health)
GET	/api/docs	Swagger UI (swagger-ui-express)
GET	/api/health	healthcheck (estado: 'OK', fecha)
POST	/api/auth/login	validateBody(loginSchema) + loginEmailLimiter + loginIpLimiter
POST	/api/auth/recuperar	validateBody(solicitarRecuperacionSchema) + recuperacionLimiter
POST	/api/auth/recuperar/validar	validateBody(validarCodigoRecuperacionSchema)
POST	/api/auth/recuperar/restablecer	validateBody(restablecerPasswordSchema)
(cualquier otra)	/api/*	404 NOT_FOUND 'La ruta solicitada no existe.'

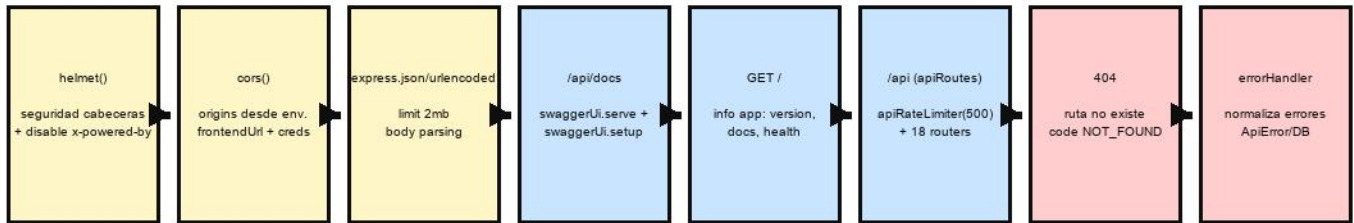
FLUJO LOGIN SIN MID (solo validacion de forma y limites de intentos)



El login NO requiere token previo; por eso NO usa authenticateToken. Tras el login, el frontend guarda gestock_token / gestock_usuario_sesion y el interceptor lo adjunta a las demas rutas (que SI pasan por el MID).

CADENA DE MIDDLEWARE DE app.ts (PUNTO DE ENTRADA UNICO)

Como se construye la aplicacion Express en backend/src/app.ts



DETALLE DE app.use('/api', apiRateLimiter, apiRoutes)

app.use('/api', apiRateLimiter, apiRoutes) -> apiRateLimiter aplica a TODO /api (max 500 / 15 min).
apiRoutes: /health y luego monta: /auth /usuarios /roles /permisos /empresas /categorias /bodegas
/productos /movimientos /recepciones /auditorias /incidencias /mantenimientos /sesiones
/notificaciones /configuracion /dashboard /reportes. Cada router define sus propios middleware.

El 404 y el errorHandler estan al FINAL (despues de apiRoutes): cualquier ruta no montada
cae en el 404; cualquier next(err) cae en errorHandler que traduce errores de Postgres
(23505 DUPLICATE_KEY, 23503 FOREIGN_KEY_VIOLATION, 23502 NOT_NULL, 22P02) y es ApiError/JSON roto.

RUTAS ANGULAR PROTEGIDAS POR GUARDS (MID CLIENTE)

app.routes.ts: canActivate + data.permission / anyPermission

Zona	Ruta Angular	Guard / Permiso
"	/	PaginaComponent (inicio publico)
auth	/auth/login	LoginComponent - publico
auth	/auth/crear-usuario	permissionGuard · permission='usuarios.create'
auth	/auth/recuperar-contrasena	RecuperacionComponent - publico
auth	/auth/sesiones-activas	permissionGuard · permission='sesiones.view'
app (layout)	app/**	permissionGuard global (LayoutComponent)
app	app/panel · app/panel/:id	permission='dashboard.view'
app	app/empresas	permission='empresas.view'
app	app/regarstrar-empresa	permission='empresas.create'
app	app/gestion/inventario	permission='inventario.view'
app	.../lista-productos	permission='productos.view'
app	.../regarstrar-productos	permission='productos.create'
app	.../bodegas	permission='bodegas.view'
app	.../categorias	permission='categorias.view'
app	app/recepcion/recepcion-mercancias	anyPermission=['recepcion.view','recepcion.create']
app	app/recepcion/historial-logistico	permission='historial.view'
app	app/gestion/auditorias	permission='auditoria.view'
app	app/gestion/roles-y-usuarios	anyPermission=['usuarios.view','roles.view']
app	app/reportes	permission='reportes.view'
app	app/configuracion	permission='configuracion.view'
app	app/programacion	anyPermission=['mantenimiento.view','mantenimiento.create']
app	app/incidencias	anyPermission=['incidencias.view','incidencias.create']
fallback	**	redirectTo "" (home-guard redirige por rol)

permissionGuard.ts: refresca la sesion (GET /auth/perfil), exige estaAutenticado(), y valida data.permission o data.anyPermission con authService.tienePermiso(). homeGuard/roleGuard redirigen a la ruta inicial por rol.

MAPA: PAGINA FRONTEND -> ENDPOINTS QUE LLAMA (CON/SIN MID)

Cada pagina Angular consulta su(s) recurso(s) via ApiService (11 llamadas directas listadas)

Componente / Pagina	Endpoints (ApiService)	Uso del MID
Login	POST /api/auth/login	SIN MID (validate+rateLimit)
Recuperar contraseña	POST /auth/recuperar · POST /auth/recuperar/restablecer	SIN authenticate
Crear usuario	GET /roles/basicos · POST /usuarios	CON MID
Sesiones activas	GET /sesiones · DELETE /sesiones/:id · POST /sesiones/cerrar-otras	CON MID
Panel	GET /dashboard/resumen · GET /empresas (estadisticas.service)	CON MID
Empresas	GET /empresas · POST /empresas · POST /empresas/seleccionar · DELETE /empresas/:id	CON MID
Registrar empresa	POST /empresas · POST /empresas/seleccionar	CON MID
Inventario	GET/POST/PUT/DELETE /productos · /productos/stock-bajo · /categorias · /bodegas	CON MID
Recepcion mercancias	GET /productos · GET /bodegas · GET/POST /movimientos	CON MID
Historial logistico	GET /movimientos · GET /recepciones	CON MID
Auditorias	GET /auditorias	CON MID
Roles y usuarios	GET/POST/PUT/PATCH/DELETE /usuarios · /roles · /roles/basicos · /permisos · /roles/...	CON MID
Configuracion	GET/PUT /configuracion · GET /sesiones · DELETE /sesiones/:id	CON MID
Mantenimiento	GET/POST /mantenimientos	CON MID
Incidencias	GET/POST /incidencias	CON MID
Reportes	GET /reportes/inventario · /reportes/movimientos · /productos · /movimientos · /bod...	CON MID
Header (layout)	GET /notificaciones · POST /notificaciones/leidas	CON MID

Todas las llamadas pasan por apiInterceptor (Bearer + manejo de errores). Las rutas publicas (login, recuperacion) no tienen token: el interceptor no adjunta Authorization.

MANEJO DE ERRORES EN EL CLIENTE (frontend)

Que hace Angular cuando la API devuelve cada código

Status	Código / Origen	Reacción del CLIENTE (MID parte 1)
200 / 201	ok/created/noContent	ApiService devuelve res.data · el componente renderiza
400 BAD_JSON	JSON invalido	interceptor -> toast(message,'error')
400 Zod/23502/22P02	DTO invalido / NOT_NULL / INVALID_PARAMETER	interceptor -> toast(message,'error')
401 UNAUTHORIZED	sin token / expirado / sesion cerrada	interceptor (si no es login): limpia sesion + toast 'Tu sesion ha expirado'...
403 FORBIDDEN	sin permiso / usuario inactivo	toast(message,'error'); en guard: toast 'No tienes permiso' + ruta inicial ...
404 NOT_FOUND	ruta o recurso no existe	toast(message,'error')
409 DUPLICATE_KEY/FOREIGN_KEY	duplicado / FK en uso	toast(message,'error')
429 RATE_LIMITED	muchos intentos	toast(message,'error'); en login.ts: 'Demasiados intentos...'
500 INTERNAL_ERROR	error interno servidor	toast(message,'error')

api.interceptor.ts: catchError -> si status 401 y no es /auth/login => limpiar sesion y redirigir; si status != 401 => toast(message,'error'). La pagina tambien hace catch local (login.ts y guards manejan mensajes especificos por código).